

ВВОДНАЯ ЛЕКЦИЯ

Основы вайб-кодинга

OpenCode, память проекта и управляемая работа с агентами

Проблема → критерии → план → маленький `diff` → запуск
→ проверка → `commit`

бесплатные инструменты • практический старт • безопасность



Комплект обновлён: 03.09.2026

Что такое вайб-кодинг

ВВЕДЕНИЕ

Разработка через естественный язык, небольшие шаги и обязательную проверку результата

Не «магия»

Модель предлагает изменения,
но не доказывает их
правильность.

+

Управляемый процесс

Проблема → критерии → план
→ diff → запуск → проверка

=

Быстрый прототип

Работает, проверяется
и может быть объяснён командой.

Что делает студент

- формулирует задачу и ограничения
- выбирает один небольшой шаг
- читает diff и запускает проверку

Что делает агент

- читает контекст проекта
- предлагает план или изменяет файлы
- помогает локализовать ошибки

Главное правило

ИИ ускоряет генерацию.

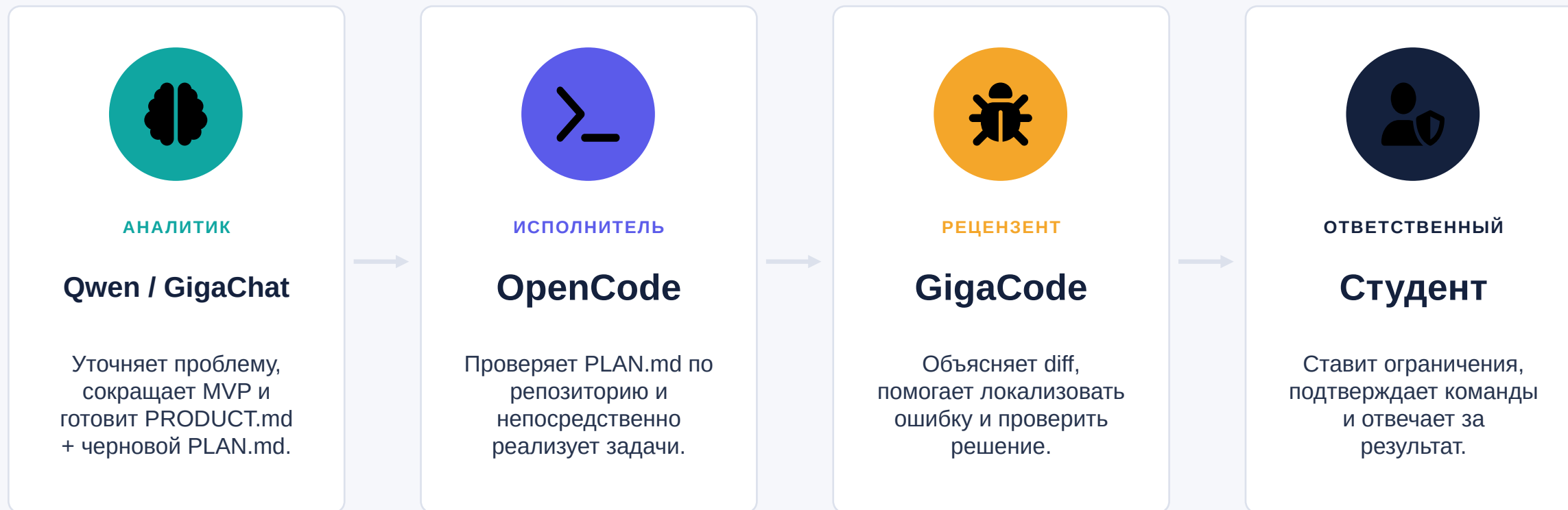
Человек отвечает за цель,
безопасность и итоговый
результат.

Вайб-кодинг на курсе: короткие итерации + память проекта + проверяемый результат.

Один конвейер — четыре роли

МОДЕЛЬ

Разделение ролей помогает не смешивать планирование, реализацию и проверку



Лучше проверять результат моделью другого производителя: она реже повторяет те же типичные ошибки. Но финальное доказательство — тест и Git diff.

Qwen Chat: сначала описываем продукт

ПЛАНИРОВАНИЕ

Бесплатный веб-чат превращает идею в PRODUCT.md и черновой PLAN.md — до запуска OpenCode

1. Исходная идея

Проблема, команда, срок, ограничения и пример пользователя.



2. Диалог с Qwen

До пяти уточняющих вопросов, сокращение MVP, рисков и лишних функций.



3. PRODUCT.md + PLAN.md

Проблема, функции MVP, критерии, этапы реализации и проверки.

ПРОМПТ ДЛЯ СТАРТА

«Ты — продуктовый аналитик. Сначала задай до пяти важных вопросов. Создай PRODUCT.md на русском: проблема, аудитория, MVP, границы и критерии.
Затем подготовь черновой PLAN.md: этапы реализации, зависимости и проверки.
Код не пиши».

Зачем отдельный чат?

- не расходует квоту модели в OpenCode;
- позволяет согласовать PRODUCT.md и PLAN.md до изменения файлов;
- оставляет токены исполнителя на код, тесты и диагностику.

*** Авторизуйтесь в Qwen заранее: без входа гостевой лимит может закончиться быстрее. Условия и лимиты могут меняться.**

Qwen Workspace и GigaChat как альтернатива

ЧАТЫ

PRODUCT.md и черновой PLAN.md готовим в чате, сохраняем в GitVerse — затем запускаем OpenCode

Q

Qwen Workspace*

- создайте отдельное рабочее пространство для проекта;
- загрузите постановку, правила, заметки, таблицы и примеры;
- попросите подготовить PRODUCT.md и черновой PLAN.md;
- не загружайте API-ключи, .env и персональные данные.

* Для Workspace, истории и устойчивой работы войдите в аккаунт.

G

GigaChat (ГигаЧат)

- используйте, если Qwen недоступен или его лимит закончился;
- дайте тот же запрос: PRODUCT.md + черновой PLAN.md;
- веб-чат удобен для русскоязычного обсуждения требований;
- сохраните результат в файлы: история чата не заменяет репозиторий.

Не путать: GigaChat — веб-чат, GigaCode — помощник в IDE.

ПЕРЕДАЧА РЕЗУЛЬТАТА В РАЗРАБОТКУ

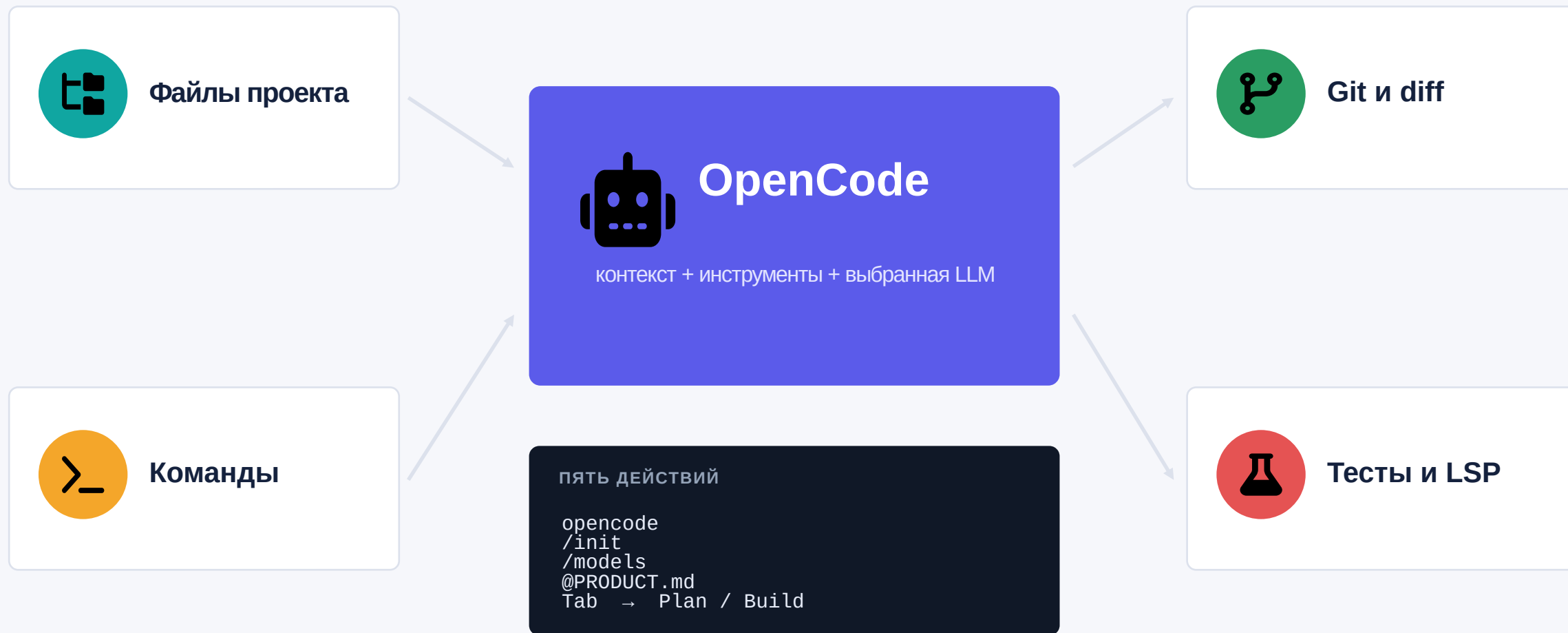


Рекомендуемый Git-сервис для курса: GitVerse. Сохраните исходное состояние и делайте commit после каждого проверенного шага.

OpenCode: агент внутри репозитория

БЛОК 1

Оболочка соединяет модель с файлами, командами, Git и диагностикой



Источник: официальная документация OpenCode — Intro, Agents, Rules, Tools.

Бесплатные модели OpenCode Zen: что выбрать

МОДЕЛИ

Список меняется — перед занятием откройте /models и убедитесь, что модель всё ещё отмечена как Free



РЕКОМЕНДУЕМЫЙ СТАРТ

Muse Spark 1.3 Contributor Free

opencode/muse-spark-1.3-contributor-free

- первый выбор для учебного кодирования и агентных задач;
- если качество или доступность нестабильны — переключитесь на MiMo-V2.5 Free;
- режим Contributor допускает использование запросов и ответов для обучения будущих моделей Meta.

Не передавайте закрытый код, персональные данные, токены и секреты.

MiMo-V2.5 Free

Сильный запасной вариант для повседневного кода и агентных задач.

Nemotron 3 Ultra Free

Сложное рассуждение и координация; для простых шагов может быть избыточен.

Nemotron 3.5 Lightning Free

Быстрые повторяющиеся действия, проверки и вызовы инструментов.

Muse Spark 1.2 Contributor Free

Предыдущее поколение; использовать как резерв при недоступности 1.3.

Ling 3.0 Flash Fin Free

Финансовая специализация; не основной выбор для общего студенческого кода.

Big Pickle

Stealth-модель без прозрачного описания — не ставить учебным стандартом.

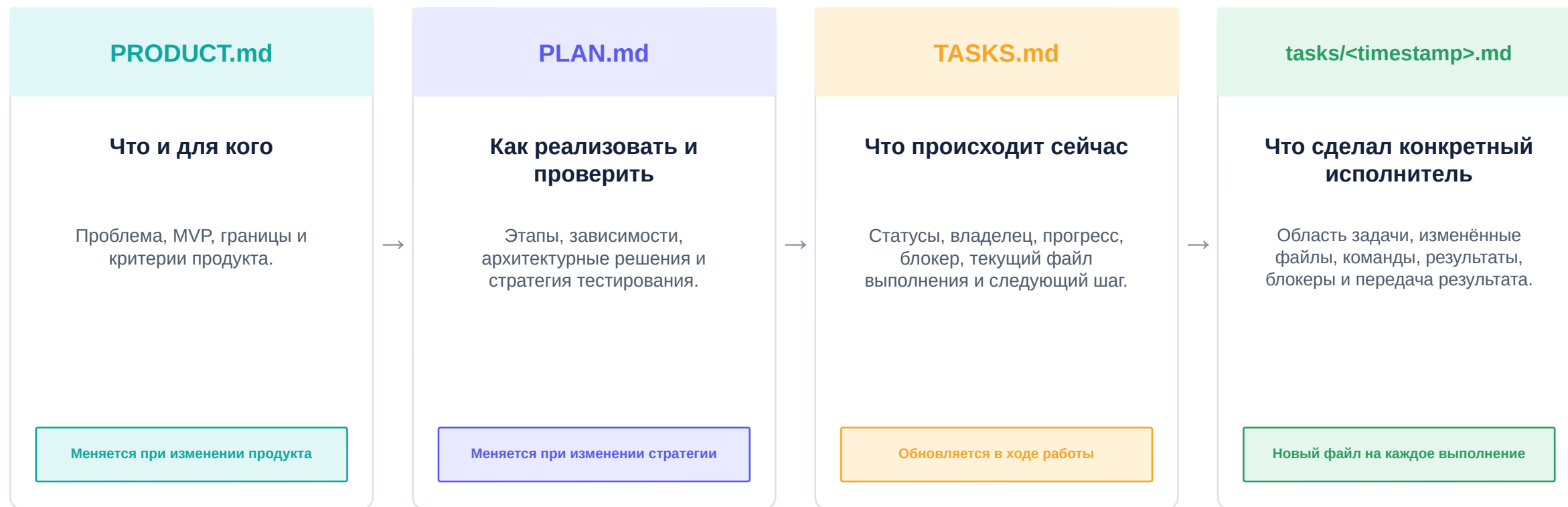
Все семь вариантов бесплатны только временно. У бесплатных endpoint могут быть собственные правила хранения данных и ограничения нагрузки.

Актуально на 03.09.2026. Источники: OpenCode Zen; Xiaomi MiMo-V2.5; NVIDIA Nemotron 3.5 Lightning.

Моя схема документов проекта

МОЯ СХЕМА

Один файл не должен одновременно быть ТЗ, архитектурным планом, журналом и памятью агента



PRODUCT.md → PLAN.md → TASKS.md → текущий tasks/<timestamp>.md → git diff + тесты

История чата помогает, но источником истины остаются файлы проекта, Git и воспроизводимые проверки.

PLAN.md: план реализации и тестирования

PLAN.MD

Стабильная стратегия до BUILD — без журнала команд и субъективных процентов готовности

PLAN.md

PLAN.md

Этап 1. Модель данных

Реализация: определить структуру Task.

Затрагивает: src/model.js, tests/model.test.js.

Проверка: unit-тесты сериализации.

Критерий: статус сохраняется и восстанавливается.

Зависит от: нет.

Этап 2. Интерфейс

Реализация: связать форму с моделью.

Проверка: основной пользовательский сценарий.

Риск: потеря данных при миграции.

Откат: сохранить совместимость старого формата.



Что должно быть в PLAN.md

- этапы и зависимости;
- затрагиваемые модули и ограничения;
- способ реализации без пошагового микрокода;
- стратегия тестирования каждого этапа;
- риски, критерии и способ отката.



Что не нужно помещать

- текущий статус и владельца задачи;
- «готово примерно на 80%»;
- полный журнал терминала и diff;
- историю переписки;
- отдельный отчёт каждой попытки выполнения.

Меняйте PLAN.md только при изменении требований, архитектурной стратегии или способа тестирования — и записывайте причину.

TASKS.md: детали реализации и текущий статус

TASKS.MD

Он превращает этапы PLAN.md в конкретные задачи и хранит точку продолжения

TASKS.md

TASKS.md

```
## T-002 — Восстановить сохранённый статус
Статус: IN_PROGRESS Владелец: storage
Этап PLAN.md: 1. Модель данных
Разрешённые файлы: src/storage.js, tests/storage.test.js
```

```
### Детали реализации
- load() должен читать сохранённое поле status;
- неизвестный status заменяется на todo;
- формат старых записей остаётся совместимым.
```

```
### Проверки: 2/4
- [x] status сохраняется;
- [x] список загружается после refresh;
- [ ] status восстанавливается;
- [ ] regression-тест проходит.
```

```
Текущая попытка: docs/agent/tasks/T-002/
20260903T101530Z_storage_a01.md
Блокер: нет.
Следующий шаг: исправить load() и запустить npm test.
```

1

Конкретизирует PLAN.md

Цель, score, разрешённые файлы, технические решения и критерии задачи.

2

Хранит текущее состояние

Статус, владелец, проверки, блокиер, ссылка на попытку и следующий шаг.

3

Один центральный владелец

TASKS.md изменяет основной агент; исполнители пишут уникальные файлы выполнения.

PLAN.md = стратегия TASKS.md = конкретные задачи + состояние

Каждое выполнение задачи — отдельный файл

TASK FILES

Временная метка + ID задачи + исполнитель предотвращают коллизии и сохраняют историю

структура проекта

```
project/
├── PRODUCT.md
├── PLAN.md
├── TASKS.md
├── docs/agent/tasks/
│   ├── T-001/
│   │   └── 20260903T091200Z_main_a01.md
│   ├── T-002/
│   │   ├── 20260903T101530Z_storage_a01.md
│   │   └── 20260903T104505Z_reviewer_a02.md
│   └── T-003/
│       └── 20260903T111010Z_main_a01.md
```

Формат внутри каталога задачи:
YYYYMMDDTHHMMSSZ_<OWNER>_aNN.md

1

Правила создания

- файл создаётся при начале выполнения, а не заранее для всех TODO;
- повтор, новая сессия или новый субагент создают новый файл;
- используйте UTC и номер попытки a01, a02...;
- если имя занято — увеличьте номер попытки.

2

Что хранит файл выполнения

- цель, разрешённый score и критерии;
- просмотренные и изменённые файлы;
- команды, результаты и доказательства;
- блокиеры и найденные риски;
- точный следующий шаг и SUBAGENT_HANDOFF;
- ссылку на предыдущую попытку той же задачи.

Субагенты не перезаписывают общий файл друг друга: каждый возвращает путь к своему уникальному timestamp-файлу.

AGENTS.md: правила маршрутизации и записи

СОСТОЯНИЯ

AGENTS.MD

Короткий корневой файл сообщает, что читать, где писать и кто владеет центральным TASKS.md

AGENTS.md

Общие правила

- Все документы, планы, статусы и handoff пишите на русском.
- Перед BUILD читайте: PRODUCT.md → PLAN.md → TASKS.md
→ файл текущей попытки.
- Только основной агент изменяет TASKS.md.
- Каждый исполнитель при старте создаёт новый файл:
docs/agent/tasks/<TASK-ID>/<UTC>_<OWNER>_aNN.md
- Субагент возвращает путь к файлу и не переписывает чужие отчёты.
- Перед изменением src/ прочитайте src/AGENTS.md.
- Перед изменением tests/ прочитайте tests/AGENTS.md.
- Не загружайте все rules и skills заранее.
- Не создавайте новый PLAN.md, если существующий остаётся корректным.

Ленивая иерархия правил

корневой AGENTS.md



src/AGENTS.md

если меняется код

tests/AGENTS.md

если меняются тесты

docs/rules/*.md

только нужная
специализация

40–80 строк: навигатор, не энциклопедия

Важно: ссылка в AGENTS.md не раскрывает файл автоматически — явно укажите условие чтения.

Перезапуск и субагенты: восстановление по фактам

ПРОДОЛЖЕНИЕ

Команда «продолжи» работает через документы, Git и тесты — не через память предыдущего чата



Новая сессия

AGENTS.md → PRODUCT.md → PLAN.md → TASKS.md → последняя попытка → git status/diff/log → тесты

1. Сравнить сохранённый статус с фактическими файлами.
2. Создать новый timestamp-файл, сославшись на предыдущую попытку.
3. Продолжить с поля «Точный следующий шаг», а не строить план заново.

TASKS.md указывает на актуальную попытку; каждый новый исполнитель создаёт новый файл, поэтому параллельные отчёты не сталкиваются.

MCP и LSP: внешние возможности и понимание кода

ИНСТРУМЕНТЫ

Подключать только те инструменты, которые реально нужны текущей задаче



MCP

Model Context Protocol подключает внешние инструменты и данные.

Примеры:

- websearch — актуальный веб-поиск;
- Context7 — документация библиотек;
- grep.app — примеры в открытом коде;
- CodeGraph — граф файлов, символов и вызовов; помогает увидеть архитектурные связи и влияние изменений.

Риск: каждый сервер добавляет контекст, разрешения и внешнюю зависимость.



LSP

Language Server Protocol даёт структурированные сведения о локальном коде.

Помогает:

- найти определение и ссылки;
- увидеть типы и диагностику;
- безопаснее переименовать символ;
- обнаружить синтаксическую ошибку.

Ограничение: LSP не проверяет бизнес-логику и не заменяет тесты.



CodeGraph показывает связи на уровне архитектуры, LSP — точные определения, типы и диагностику. Оба инструмента дополняют, но не заменяют тесты.

Источники: Model Context Protocol; Microsoft LSP; документация OpenCode; CodeGraph.

Skills: готовые рабочие процессы

SKILLS

Их можно искать в интернете и поручать чат-модели предварительный поиск



Что даёт skill

Повторяемую методику для конкретной задачи:

- code review;
- тестирование интерфейса;
- анализ данных;
- подготовка документации;
- проверка доступности;
- безопасная работа с Git.

OpenCode видит описания и загружает полный skill по необходимости.

Как искать



Описать повторяющуюся задачу



Попросить чат найти реальные репозитории



Сравнить лицензию, scripts и разрешения



Выбрать один непересекающийся skill



Не тащить всё подряд

Большая коллекция создаёт:

- дублирование;
- противоречивые инструкции;
- сложный выбор;
- лишние зависимости;
- устаревшие сценарии;
- более широкую поверхность атаки.

Для небольшого хакатона обычно достаточно нескольких чётко различающихся skills.

Skill может содержать исполняемый код

БЕЗОПАСНОСТЬ

Относитесь к нему как к сторонней программе, а не как к красивому промпту

СТРУКТУРА SKILL

```
my-skill/
├── SKILL.md      инструкции
├── scripts/      код
├── references/   справка
├── assets/       шаблоны
└── ...
```



Что он может сделать

Читать файлы и env, устанавливать пакеты, обращаться в сеть, менять Git, удалять или отправлять данные.

Проверка перед установкой

- ✓ Источник, автор, лицензия и дата обновления
- ✓ SKILL.md, scripts и другие исполняемые файлы
- ✓ Shell-команды, установки и сетевые запросы
- ✓ Доступ к файлам, токенам и переменным окружения
- ✓ Минимально необходимые разрешения
- ✓ Первый запуск в тестовом репозитории или контейнере

Красные флаги: `curl | bash · sudo · rm -rf · printenv · force push`



Ответ чат-модели — предварительный аудит, а не сертификат безопасности.

Agent Skills specification: каталог может включать scripts/ с исполняемым кодом и другие ресурсы.

Бесплатный учебный стек: проверять заранее

ДОСТУП

Планирование отделяем от исполнения, а состояние обязательно сохраняем в GitVerse



Qwen Chat / GigaChat

PRODUCT.md и черновой PLAN.md без расхода обращений модели-исполнителя. В Qwen войдите заранее.

ПЛАНИРОВАНИЕ



OpenCode Zen

Кодовый агент и текущие бесплатные модели. Перед занятием проверьте /models.

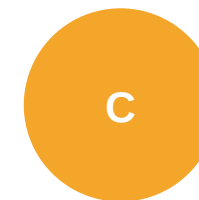
РЕАЛИЗАЦИЯ



GitVerse

Удалённый репозиторий курса: история commits, совместная работа и восстановление состояния.

ПАМЯТЬ



GigaCode

Review, объяснение diff и отладка в поддерживаемой IDE.

ПРОВЕРКА

Qwen: авторизуйтесь заранее. OpenCode Zen: бесплатные модели временные и могут использовать данные по собственным условиям — не передавайте секреты.

Актуально на 03.09.2026. Источники: OpenCode Zen, Qwen Chat, GigaChat, GitVerse и GigaCode.

Как выбрать модель и не исчерпать бесплатный доступ

ПРАКТИКА

Основное планирование делаем в веб-чате; в OpenCode запускаем только небольшие проверяемые шаги

Первое подключение

```
/connect  
→ OpenCode Zen
```

```
/models  
→ Muse Spark 1.3 Contributor Free  
→ MiMo-V2.5 Free  
→ Nemotron 3.5 Lightning Free
```

Рабочий цикл:

Qwen/GigaChat: PRODUCT.md + PLAN.md

OpenCode: одна задача → локальная проверка → commit

1. Основной выбор

Muse Spark 1.3 Contributor Free — кодирование и agentic tasks.

2. Сильный резерв

MiMo-V2.5 Free — повседневный код, длинные задачи и агентная работа.

3. Быстрый исполнитель

Nemotron 3.5 Lightning Free — рутинные команды, проверки и инструменты.

Esc останавливает ненужный цикл. Не просите переписывать весь проект и не повторяйте запрос без новой информации.

«Free» означает нулевую цену сейчас, но не обещает постоянную доступность, отсутствие ограничений или конфиденциальность данных.

Проверено 03.09.2026. Перед каждым потоком заново откройте /models и официальную страницу OpenCode Zen.

Пять правил на всю неделю

ФИНАЛ

Пусть они останутся на последнем экране и в памятке

1

Сначала критерии, потом код

Понятно, что именно считается готовым.

2

Одна итерация — одна задача

Маленький diff легче проверить и отменить.

3

Ответ AI — не доказательство

Доказательства: запуск, тест, лог и Git diff; история хранится в GitVerse.

4

Состояние хранится в проекте

PLAN.md хранит стратегию; TASKS.md и timestamp-файлы сохраняют точку продолжения.

5

Минимум разрешений и доверия

Проверяем skills, MCP, команды и секреты.

Не коммитьте код, который никто в команде не может объяснить.

Приложение: пром프트 «Продолжи»

ПРИЛОЖЕНИЕ

Полная версия находится в архиве в `prompts/03-continue.md`

`prompts/03-continue.md`

Продолжи выполнение сохранённого плана проекта.
Сначала ничего не изменяй. Все документы и статусы пиши на русском языке.

1. Прочитай `AGENTS.md`, `PRODUCT.md`, `PLAN.md` и `TASKS.md`.
2. Найди задачу `IN_PROGRESS` и открой указанный в `TASKS.md` файл последней попытки.
3. Проверь `git status`, `git diff`, последние `commits` и относящиеся к задаче тесты.
4. Сравни сохранённый статус с фактическим состоянием файлов.
5. Сообщи: что завершено, что осталось, blockers и точный следующий шаг.
6. Перед продолжением создай новый файл:
`docs/agent/tasks/<TASK-ID>/<UTC>_<OWNER>_aNN.md`
и укажи в нём ссылку на предыдущую попытку.
7. Продолжи с поля «Точный следующий шаг».

Не создавай новый `PLAN.md`, если требования и стратегия остаются корректными.
Центральный `TASKS.md` обновляет только основной агент.

Приложение: источники и состав комплекта

ПРИЛОЖЕНИЕ

Все внешние условия проверены на 3 сентября 2026 года



Официальные источники

- opencode.ai/docs — OpenCode, Zen, agents, MCP, LSP и skills;
- opencode.ai/docs/zen — текущие бесплатные модели и правила данных;
- mimo.xiaomi.com/mimo-v2-5 — агентные и кодовые возможности MiMo-V2.5;
- developer.nvidia.com — Nemotron 3.5 Lightning и agentic execution;
- modelcontextprotocol.io — MCP; [codegraph docs/GitHub](https://codegraph.docs/GitHub) — граф кода и MCP-сервер;
- microsoft.github.io/language-server-protocol — LSP;
- agentskills.io — структура skills и scripts;
- qwen.ai, [GigaChat](https://gigachat.me), [GitVerse](https://gitverse.com) и [GigaCode](https://gigacode.com) — инструменты курса.



Файлы для преподавателя и студентов

- презентация PPTX и PDF;
- памятка DOCX и PDF;
- ZIP с минимальным проектом;
- отдельный AGENTS_examples_RU.md;
- PRODUCT.md, PLAN.md и TASKS.md;
- timestamp-шаблон файла выполнения;
- промпты создания плана и продолжения;
- русскоязычные модульные AGENTS.md;
- пример безопасной конфигурации OpenCode;
- тестовый минимальный web-проект.



Перед каждым новым потоком перепроверьте список Free в OpenCode Zen, условия обработки данных, регистрацию и версии инструментов.